Incident Triage RAG with LangChain

Colin Vitols | IT7075 Applied AI for Cybersecurity | November 2, 2026

Project folder: GitHub - langchain-rag-project / langchain_project

Video URL - paste the shared recording link into the field below before submission.

Concept check

What problem does LangChain solve

Calling a model API only handles the model request. LangChain supplies common interfaces for documents, retrieval, prompts, models, and output parsing, so I can connect those stages without maintaining all of their calling code myself.

Why use a prompt template

A prompt template is reusable text with named input fields. The context and question placeholders let me keep the instructions consistent while changing the evidence and question for each request, rather than rebuilding a hard-coded prompt every time.

What the prompt model parser chain does

The prompt fills its placeholders and creates the messages; the model consumes those messages and returns a response. The output parser turns that response into the expected type, which is a string here. My offline model stage quotes context instead of generating new reasoning.

How a retriever fits into a chain

A retriever takes a question and returns relevant Documents, including their metadata. In this chain, its output becomes the context while a passthrough sends the original question to the other prompt placeholder.

When to branch and when to run in parallel

A branch selects a path based on a condition, such as routing a critical incident to an urgent review workflow and other incidents to routine triage. Parallel steps both run on the same input, such as independently summarizing endpoint and network evidence before combining the findings.

The rebuild and what it replaced

I rebuilt the incident-triage RAG using the same five Markdown documents and the same eight questions from the earlier project: six answerable and two unanswerable. All copied files, including the source notes and goldset.json, are checked byte-for-byte against rag_project; data_manifest.json records their SHA-256 hashes. The five indexed documents contain 10,204 characters, and SOURCES.md is excluded from retrieval just as before.

The notebook first completes all five practice stages on the course knowledge base, then builds the custom bot. The custom pipeline uses TextLoader, CharacterTextSplitter, normalized all-MiniLM-L6-v2 embeddings on CPU, cosine Chroma retrieval, ChatPromptTemplate, and LCEL pipes. A deterministic offline model returns a labeled excerpt or an empty-context refusal, which is the permitted no-key path.

LangChain stage	Earlier code replaced
TextLoader and CharacterTextSplitter	load_and_chunk() and split_into_chunks()
HuggingFaceEmbeddings and Chroma.from_documents()	embed() and build_collection()
as_retriever() with score threshold	retrieve() and its distance-filtering loop
ChatPromptTemplate	SYSTEM.format(...) inside ask()
LCEL dictionary and pipe sequence	The orchestration inside ask()

Not every helper disappears. format_sources() still controls how citations enter the prompt, and the offline RunnableLambda replaces grounded_answer() without turning it into a real language model. The value of LCEL is that the assembled flow has one invoke interface and clearly named inputs.

Where the framework helped and hid details

Document objects preserve source metadata through splitting and retrieval, and the standard retriever fits directly into the chain. That reduces repetitive embedding, storage, and dispatch code. However, a one-line store constructor hides index configuration and score conversion; the splitter also chooses exact boundaries and when to stop emitting overlapping tails.

Those hidden choices matter when comparing experiments or investigating a missed security instruction. I made the distance metric explicit, inspected raw distances against relevance scores, and compared both retrievers on identical chunks. Collection names are unique on every build, so rerunning cells cannot silently accumulate old chunks.

Measured comparison

Both implementations used the same normalized MiniLM model, cosine distance, and evidence-matching rule: a gold phrase must occur within one retrieved chunk. Found counts the six answerable questions that satisfy that rule; refused counts the two unanswerable questions that return no chunks. These measure retrieval behavior, not generated-answer accuracy.

The default has 500-character chunks, 80-character overlap, and k=3. The manual rule keeps distance at or below 0.50. The LangChain rule keeps relevance at or above 0.50, using relevance = max(0, min(1, 1 - cosine distance)). For every positive cutoff tested, the rules are equivalent. A stricter distance of 0.35 therefore becomes a higher relevance cutoff of 0.65, not 0.35.

Change from default	Distance / score	Chunks M / L	Found M / L	Refused M / L
Default	0.50 / 0.50	26 / 26	6/6 | 6/6	2/2 | 2/2
Chunk size 350	0.50 / 0.50	41 / 39	5/6 | 5/6	2/2 | 2/2
Chunk size 700	0.50 / 0.50	20 / 20	4/6 | 4/6	2/2 | 2/2
k = 1	0.50 / 0.50	26 / 26	5/6 | 5/6	2/2 | 2/2
k = 5	0.50 / 0.50	26 / 26	6/6 | 6/6	2/2 | 2/2
Stricter threshold	0.35 / 0.65	26 / 26	2/6 | 2/6	2/2 | 2/2
Looser threshold	0.65 / 0.35	26 / 26	6/6 | 6/6	1/2 | 1/2
Loosest threshold	0.80 / 0.20	26 / 26	6/6 | 6/6	1/2 | 1/2

M = manual implementation; L = LangChain. Every row changes at most one setting from the default; overlap stays at 80 characters. The labels in results_langchain.csv preserve the earlier distance labels, while the explicit score_threshold column gives the LangChain value.

The default achieves 6/6 found and 2/2 refused in both versions. All eight manual configurations were rerun from the earlier notebook functions and reproduce the committed results. With identical default chunks, both retrieval interfaces return the same accepted text for all eight questions; this is stronger evidence than comparing totals alone.

At chunk size 350, the manual splitter emits 41 chunks while LangChain emits 39. The two omitted manual chunks are redundant final tails already contained in retained chunks; the notebook checks this directly. Their omission does not change the measured outcomes. At the default, both versions have 26 chunks.

A stricter cutoff reduces found to 2/6. Looser cutoffs keep 6/6 found but allow unsupported Q8 through, reducing refusals to 1/2. Because the representations and thresholds are controlled, the framework itself does not improve the measured retrieval quality.

Failure analysis and next choice

Q3 at k equals one

The question asks: “What should an analyst preserve from a suspected phishing email?” At k=1, the first email_triage.md chunk scores 0.5963 and does not contain the exact gold phrase. The second chunk scores 0.5926 and contains “preserve the original message with full headers and attachment hashes,” but k=1 excludes it.

The retrieved first passage says to preserve the original message or an approved export so headers, attachment hashes, URLs, and routing details remain available. It is relevant and overlaps semantically with the expected action. Therefore, this is specifically an exact-evidence metric miss: it would be misleading to describe it as retrieving an unrelated document or proving a wholly wrong natural-language answer.

The small score gap explains the ranking problem. Raising k to the default of three retains the second passage and restores Q3 evidence. In a larger system I would examine chunk boundaries and a semantic answer-quality check, rather than concluding from one missing phrase that the response is unusable. I would keep the fixed lab metric for this comparison and evaluate any new metric separately.

The unsupported policy question

Q8 asks for the exact SIEM retention period required by the program, which the documents do not specify. At looser thresholds, related incident-policy text is retrieved anyway. The offline model quotes whatever context it receives, so a nonempty retrieval can produce a [RAG] response even though the answer is absent. This is why semantic similarity alone is not an answerability guarantee.

Which version I would build next time

For a small maintained application, I would use LangChain because the common interfaces and metadata handling make the pipeline easier to extend. I would still retain tests and diagnostics for chunk boundaries, metric selection, and refusal behavior. For teaching the retrieval mechanics or investigating a scoring discrepancy, the manual version is easier to trace because each embedding, query, and filtering step is visible.

Limits of the evidence

This run uses a cached local embedding model on CPU and an offline excerpt renderer, with no live generative model, API key, or GPU. Evidence found across the retrieved set does not guarantee the displayed excerpt includes the full answer. The eight authored questions are a small smoke test, not a held-out benchmark, and no latency or developer-effort benchmark was performed.

Sources and reproducibility

Assignment: 05_LangChain/assignments/project/MiniProject_FrameworkRAG.pdf. Baseline: rag_project at commit 4bd86df17707d1a43831d1067c6555b891137002. Results, per-question diagnostics, transcript, hashes, and package versions are saved in the linked project folder. The accompanying live-recording guide shows how to restart the kernel and execute the notebook during the video.

LangChain Chroma API reference and CharacterTextSplitter API reference. Implementation behavior was also checked in the installed packages and executed assertions.